Multi-agent II: When Coordination Fails

The failure taxonomy; adversarial validation in three tiers; locks; cost-aware routing; delegation, handoff, and the contracts a marketplace would need

SURVEY | BUILD | GOVERN | FALSIFY

CIDM 6333 · Agentic Systems Design and Development
Survey: Cemri et al. (MAST, 2025) · Anthropic's multi-agent research system · A2A v1.0 delegation · AP2 mandates · Build: a second-family adversary, a lock with a TTL, a route table with a budget

Two hours, eight moves

The run of show

1	0:02	**Framing**	A subagent's claim is not evidence; a receipt is
2	0:10	**Spine**	The taxonomy; claim, receipt, verdict; adversarial tiers; locks; routing and cost; delegation as a contract
3	0:50	**Common mistake**	The reviewer read the summary, not the work
·	0:55	**Break**	Five minutes
4	1:00	**Teardowns**	MAST (Cemri et al.); Anthropic's system; a live Design Analysis; 'Don't build multi-agents'; A2A v1.0; AP2
5	1:30	**Reference design**	Summary laundering (05-24); the repudiations (June); the undispatched Stage 2 (08-21)
6	1:45	**Lens and card**	A strong Sortie 9; Flight Card 9 brief (corner); each card names its planted falsehood
7	1:54	**Lab and Sprint**	Self-guided lab; the Sprint, the async Go/No-Go, the Chase

A subagent's claim is not evidence

Adding agents adds authors of claims; it does not add evidence

A worker saying "all tests pass" produced a sentence. The harness that ran the tests produced evidence.

Two rules the spine returns to

Rule one. A claim enters the flight log only with a harness-written receipt: command, exit code, commit, output hash.

Rule two. The check comes from different blind spots: a second model family, then an independent subagent, then a human.

Tonight's claim, which Sortie 9 will try to disprove

A subagent's claim never becomes evidence without a receipt; a second-family adversary refutes a planted false claim.
Falsifiers: a flight-log claim with no receipt; NOT-REFUTED on the planted falsehood; a lock held by two workers or changed with no ledger event.

The taxonomy: where multi-agent systems fail

Cemri et al. (2025): 14 failure modes in 3 categories, from 1,600+ annotated traces across 7 frameworks

FC1 · System design (44%)

Failure in the setup: task or role spec, stopping condition, memory.

Step repetition 15.7%

Unaware of stopping conditions 12.4%

Disobey task specification 11.8%

Fix location: your prompts, roles, halt rules

FC2 · Inter-agent misalignment (32%)

Failure in the traffic between agents: information lost, ignored, never requested.

Reasoning-action mismatch 13.2%

Task derailment 7.4%

Fail to ask for clarification 6.8%

Fix location: contracts and handoffs

FC3 · Task verification (24%)

Failure at the end: the check was missing, wrong, or early.

Incorrect verification 9.1%

No or incomplete verification 8.2%

Premature termination 6.2%

Fix location: receipts and the adversary

gzkit adds two names: summary laundering (a summary promoted to fact) and hallucinated authorization (Appendix C #7: acting on a gate nothing records).

Claim, receipt, verdict, attestation

Four objects with four authors; only two of them are evidence

Object	Who writes it	What it proves	Where it may go
Claim	A worker model, in its result (HandoffResult status plus text: “DONE; 3/3 tests pass”)	That the worker produced this sentence. Nothing about the code.	The ledger, marked unverified. Never the flight log.
Receipt	The harness, by running the command itself: cmd, exit code, commit, output hash, time	What the command printed and returned, at that commit.	receipts/ and the ledger. The flight log cites it by path.
Adversary verdict	A second model, given claim and receipt: REFUTED or NOT-REFUTED, citing a field	Whether a model with other blind spots could break the claim. Itself a claim.	The ledger, with model id and family. The Chase checks its citation against the receipt.
Attestation	A human, by name, after reading the receipts	That a person judged the work and answers for it.	Gate 5. Never inferred, never written by an agent on the human's behalf.

HandoffResult statuses from Sortie 8: DONE | DONE_WITH_CONCERNS | NEEDS_CONTEXT | BLOCKED. The verdict does not upgrade a claim; the receipt does.

NOT-REFUTED means “this adversary could not break it with this evidence.” It never means “verified.”

Adversarial validation in three tiers

Independence is the property you are buying; size is not

The tiers (chapter 18)

Tier 1 · cross-vendor. A different family, different blind spots. Evaluators favor their own output (Panickssery, Bowman, & Feng, 2024).

Tier 2 · independent subagent. Same family, fresh context, shared blind spots.

Tier 3 · human. The Chase, or you. Only this tier attests.

What this costs you on OpenCode

Worker on Big Pickle: adversary on an OpenCode Go model, named family (GLM, DeepSeek, Kimi, Qwen, MiniMax, MiMo).

Big Pickle's family is undisclosed: write that; run two Go families.

No Go subscription: a free Zen model (Nemotron, MiMo). One family is “degraded mode”.

The adversary gets the claim and the receipt. Never the worker's summary, reasoning, or the planted answer.

Worked config: the adversary, the receipt, the ledger line

An OpenCode agent file, a harness-written receipt, and the event that records the verdict

```
# .opencode/agents/adversary.md     (model is a different family from the worker's)
---
description: Tries to refute one claim using only the evidence attached. Never edits, runs, or fetches.
mode: subagent
model: opencode-go/glm-5.3-flash          # worker runs on opencode-go/deepseek-v4-flash
permission:
  edit: deny
  bash: deny
  webfetch: deny
---
You receive CLAIM and EVIDENCE. Try to show the CLAIM is false using only EVIDENCE.
Reply with one JSON object and nothing else:
{"verdict": "REFUTED" | "NOT-REFUTED", "cites": "<field or line in EVIDENCE>", "missing": "<what you lack>"}
NOT-REFUTED means you could not refute the claim with this evidence. It does not mean the claim is true.

# receipts/2026-10-31T19-02-11_unittest.json          (written by the harness, never by a model)
{"cmd": "python -m unittest -v", "exit_code": 1, "tests_run": 3, "failures": 1,
 "commit": "a1b2c3d", "stdout_sha256": "9f2c…", "ran_at": "2026-10-31T19:02:11Z"}

# ledger event the harness appends after the adversary answers
{"event": "adversary_verdict", "claim_id": "c-017", "model": "opencode-go/glm-5.3-flash",
 "family": "GLM", "tier": 1, "verdict": "REFUTED", "cites": "receipts/…_unittest.json#failures"}

# from the shell: opencode run --agent adversary -f claim.md -f receipt.json "Refute the CLAIM."
```

Locks: who may work on what

A token-block lock with a TTL, and the ledger as its witness

1 → 2 → 3 → 4

Acquire atomically

Create locks/<resource>.lock with os.open(path, O_CREAT | O_EXCL). Exactly one caller succeeds. Write {holder, token, acquired_at, ttl_s}.

Ledger: lock_acquired.

Deny the second

A second worker's create fails with FileExistsError. No waiting, no overwrite. It returns BLOCKED naming the holder.

Ledger: lock_denied.

Release with the token

Only the token holder may delete the lock. A wrong token is refused.

Ledger: lock_released.

Expire, then reclaim

If now > acquired_at + ttl_s, the holder is presumed dead. Record the expiry, then take the lock.

Ledger: lock_expired, then lock_acquired.

The falsifier is not "two workers collided." It is a lock that changed hands with no ledger event.

Routing and cost: send each task to a tier

Route by task class, declare a budget per sortie, and record the cost of every call

Task class	Tier	Example model (verify)	Listed price in / out per 1M tokens	Why this tier
Classify, extract, format	small	opencode-go/glm-5.3-flash	$0.15 / $0.50	A schema check catches the errors; errors are cheap.
Implement a small change	mid	opencode-go/deepseek-v4-flash	$0.15–0.30 / $0.60–1.20 (peak/off-peak)	The receipt checks the result.
Plan, review, settle a conflict	large	opencode-go/kimi-k3	$3.00 / $15.00	An error here reaches every worker. Kimi K3: about 110 requests per 5 hours on Go.
Adversary	other family	a family the worker is not	small tier is enough	Independence matters more than size.
Free or local	—	opencode/big-pickle; Ollama	$0	Big Pickle data may be used to improve the model; local costs your laptop's time.

Prices and limits from opencode.ai/docs/go, checked 09/24/2026. Go is $10/month with per-model limits per 5 hours, week, and month (20% / 50% / 100%).

RouteLLM (Ong et al., 2024): 85% lower cost on MT Bench at 95% of GPT-4's quality. FrugalGPT (Chen, Zaharia, & Zou, 2023): up to 98% lower cost. Your route table is the small version.

Delegation is a contract: what a handoff must carry

Four things cross the boundary when one agent gives work to another; authority is not one of them

Identity

Who asks and who answers, by name and model id. A2A v1.0 signs Agent Cards; your dispatch record names worker and model.

Scope

What the worker may touch: paths, tools, its lock. Permissions come from config, never from the task text.

Result contract

The shape of what comes back (HandoffResult; ReviewResult) and the evidence each status requires. DONE without a receipt parses as NEEDS_CONTEXT.

Evidence obligation

Who produces the receipt and where it lands. A worker-written receipt is a claim with a file extension.

Authority does not cross: the user's approval stays with the user's host. A peer asks for it (A2A input-required, auth-required).

The common mistake: the reviewer read the summary

Five ways a Sortie 9 passes without testing anything

> *The reviewer approved the summary. Nobody reviewed the work.*

Narrative, not receipt. A story always coheres.

Same model, new prompt. That is tier 2; say so.

Answer leaked. The test then measures reading.

Refutes everything. No true-claim control, no proof (test point 4).

NOT-REFUTED logged as "verified." A verdict is a claim.

Break · 0:55–1:00

Five minutes

Your worker says “all tests pass.” What must you see before writing that in your flight log, and who wrote it?

Teardown: MAST, the fourteen modes

Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" (arXiv 2503.13657; NeurIPS 2025) · prevalence across MAST-Data

FC1 · System design	%	FC2 · Inter-agent misalignment	%	FC3 · Task verification	%
1.1 Disobey task specification	11.8	2.1 Conversation reset	2.2	3.1 Premature termination	6.2
1.2 Disobey role specification	1.5	2.2 Fail to ask for clarification	6.8	3.2 No or incomplete verification	8.2
1.3 Step repetition	15.7	2.3 Task derailment	7.4	3.3 Incorrect verification	9.1
1.4 Loss of conversation history	2.8	2.4 Information withholding	0.85		
1.5 Unaware of stopping conditions	12.4	2.5 Ignored other agent's input	1.9		
		2.6 Reasoning-action mismatch	13.2		
Total	44.2	Total	32.4	Total	23.5

Figures from the paper's HTML (v3, 26 Oct 2025). 1,642 traces from 7 frameworks: ChatDev, MetaGPT, HyperAgent, AppWorld, AG2 (MathChat), Magentic-One, OpenManus. Human κ = 0.88; LLM annotator κ = 0.77.

Appendix C map: 2.6 neighbors #3 fabrication; 3.2 is #4 skipped cheap verification; 3.3 is summary laundering.

MAST, read for mechanism

What the authors did about it, what it cost, and what it could not fix

Mechanism and design choice

Mechanism. Most failures start in the setup: a vague role, no stopping rule, a misaimed check.

Design choice. A grounded-theory taxonomy from 150 traces, then an LLM annotator to scale labeling.

Interventions (ChatDev). Clearer role specification: +9.4% success. High-level verification step: +15.6% on ProgramDev.

Cost, failure mode, and your build

Cost. Labeling is expensive; the LLM annotator is cheaper and agrees less (0.77 against 0.88).

Failure mode left. Final-stage, low-level checks alone are inadequate; better prompts did not remove categories.

For Sortie 9. Receipt, then adversary; neither replaces the other. Code one failure with MAST.

Appendix C: #4 skipped cheap verification and #9 metagaming the gate are what a final-stage check alone invites.

Teardown: Anthropic's multi-agent research system (June 2025)

An orchestrator-worker system in production, read for what coordination cost

15×

tokens used by the multi-agent system relative to a chat (single agents: about 4×)

80%

of performance variance on BrowseComp explained by token usage alone

90.2%

better than single-agent Claude Opus 4 on Anthropic's internal research eval

Mechanism. A Claude Opus 4 lead starts Claude Sonnet 4 subagents searching in parallel; a CitationAgent ties claims to sources.

Failures reported. Duplicated searches and over-spawning, fixed with detailed task descriptions and effort rules. Synchronous subagents: one slow one blocks all.

Multi-agent fits breadth-first research and fits badly where all agents share one context, including most coding. Name your system's case.

Live Design Analysis: Anthropic Research (2025)

Built on the board from the engineering post alone; the class argues the "why" row

Component	Answer (from the June 2025 post)
Foundation	Hosted. Claude Opus 4 lead; Claude Sonnet 4 subagents.
Perception	The user's query; search and tool results; subagents' condensed findings.
Planning & reasoning	The lead decomposes the query and plans; subagents plan their own searches.
Tools & orchestration	Orchestrator-worker; parallel subagents with search tools; a CitationAgent after the report.
Memory & context	Plan saved to memory; context past 200,000 tokens is truncated; outputs to a filesystem.
Coordination	Synchronous: the lead waits for each set of subagents. Early: duplicated searches, too many subagents.
Evaluation & feedback	LLM judge with a rubric (factual accuracy, citation accuracy, completeness); about 20 real queries; human testers.
Governance & human	Board question: where does a human stand between a subagent's finding and the report?
Runtime & operations	Long-running and stateful; rainbow deployments; about 15× chat tokens.

Judgment loop · Decision: parallel subagents · Privileged: breadth, speed · Cost: ~15× tokens · Failure mode: relayed findings as fact (FC2/FC3) · Preservation: citations

Counterpoint: “Don't Build Multi-Agents” (Cognition, June 2025)

Walden Yan's two principles, from the team that builds a coding agent

> *“Share context, and share full agent traces, not just individual messages.”*
> *“Actions carry implicit decisions, and conflicting decisions carry bad results.”*

Mechanism. Two subagents on one task make decisions the other cannot see.

Design choice. One single-threaded agent; compress history when the window fills.

Against Anthropic's post: no conflict. Research splits cleanly; coding carries decisions forward. Your adversary shares nothing with the worker.

Teardown: A2A v1.0 and delegation

Agent2Agent reached 1.0.0 on 12 Mar 2026 (1.0.1 on 28 May); Linux Foundation project; 150+ organizations by April

What a delegation is

A client agent reads the remote Agent Card (identity, skills, endpoint, auth) and sends a task. States: working, input-required, auth-required; then completed, failed, canceled, or rejected. Results return as Artifacts made of Parts.

What 1.0 added

- Signed Agent Cards
- tasks/list with filtering and paging
- OAuth 2.0 device-code and PKCE flows
- Multi-tenancy; protocol separated from transport bindings
- SDKs in Python, JavaScript, Java, Go, .NET

What stays opaque

The spec: agents collaborate “without needing access to each other's internal state, memory, or tools.” You never see the peer's receipts. A signed card proves the sender, not the truth of the artifact.

Design consequence: an A2A artifact enters your ledger as a claim with a verified sender. Re-run what you can; send the rest to your adversary.

Teardown: what an agent marketplace would need

Chapter 18's three requirements, plus the one the protocols have started to build

Identity

Who is the agent, and who operates it. Partly built: A2A signed Agent Cards.

Authorization

Proof that a human granted this agent this scope. AP2 (announced Sep 2025) uses mandates: signed, tamper-evident credentials for checkout and payment.

Attestation of work

Proof that the work was done as claimed. Not built in any protocol: no standard receipt for “tests pass” crosses an agent boundary.

Liability

Who answers when the agent is wrong. AP2 names accountability; the answer is still contracts between people.

The protocols prove who sent a claim and what the human allowed. Proving the work was done is still your harness's job.

Reference design: the ten-deep chain (2026-05-24)

Summary laundering: each layer's polished summary became the next layer's fact

A subagent's claim is not evidence.

What happened. Ten subagents deep; summaries became the layer above's facts.

What resulted. The DO IT RIGHT rules.

MAST reading. FC2, FC3; Appendix C #3 fabrication.

Reference design: the repudiations (June 2026)

Gate 5 attributed to a human who never gave it; attestations for work that did not exist

The episodes (Appendix G)

2026-06-13. A Gate-5 attestation attributed to an operator who never gave it. ADR-0.0.71.

2026-06-16. Four briefs attested for nonexistent components.

Chapter 18 counts seven Gate-5 repudiations in June.

The mechanism, and yours

Failure modes. #7 hallucinated authorization (June 13); #3 fabrication (June 16). Each stage trusted the last.

What resulted. gz obpi repudiate appends a repudiation; the attestation stays.

For your system. Append a correction to a refuted PASS; never edit the old line.

An attestation an agent wrote for a human is not an attestation. Record the correction; keep the error visible.

Reference design: the undispatched Stage 2 (2026-08-21)

Presence is not procedure: a marker file licensed work nobody dispatched or reviewed

What happened

The gate checked only that a marker file existed. A stale marker licensed about 350 lines, no dispatch, no review. Stage 2 (implement) never ran.

What resulted

The rule: “A presence check answers ‘is something armed’, never ‘did the procedure run’.” The gate must read the record of the procedure.

Your build borrows

- gz obpi dispatch: a dispatch record per task (“credit is never inferred”)
- gz obpi lock: who may work on what
- DISPATCH_MODEL_MAP: routing by role
- Step 4b: Codex preferred as tier-1 adversary; one vendor = degraded mode

Test point 1 is the same check: a claim with no receipt is a marker file.

Evaluation lens: a strong Sortie 9

The Chase reads the sealed falsehood, the receipts, and the ledger slice, and nothing else

Weak

- Worker's model with a “critic” prompt, written up as cross-family
- Adversary handed the summary, not the receipt
- “Tests pass” copied into the flight log
- NOT-REFUTED written as “verified”
- In-process dict lock; no TTL; no ledger events
- Cost reported as “cheap”; no per-call numbers

Sound

- Falsehood named before Go; adversary never told
- Harness receipt: command, exit code, commit, hash
- Named second family; REFUTED cites a receipt field
- True-claim control returns NOT-REFUTED
- Atomic lock with TTL; each change in the ledger
- Route table in config; cost per call under budget

Exceptional

- Two planted falsehoods, one found only in code
- Tier 1 and tier 2 on one claim; verdicts compared
- Holder killed mid-task; expiry and reclaim logged in order
- Router drops a tier at budget; switch logged
- One MAST mode reproduced on purpose, coded in debrief

Flight Card 9 brief · the planted falsehood

Corner of the envelope · depends on Sortie 8 PASS · Go is the Chase's signature · every card names its planted falsehood

```
ENVELOPE            corner
DESIGN CLAIM        A subagent's claim never becomes evidence without a receipt, and a second-family
                    adversary refutes a false claim planted in a worker's result.
SYSTEMS UNDER TEST  worker + reviewer (Sortie 8) · receipt writer · adversary (family named) · lock table
                    · route table + budget · ledger
SUBSTRATE           repo + commit · worker model id + family · adversary model id + family · budget $___
DEPENDS ON          Sortie 8 PASS (HandoffResult contract; a FAIL verdict blocks)
PLANTED FALSEHOOD   named here, hidden from the adversary (e.g. worker reports "3/3 tests pass"; one fails)

TEST POINTS
  1  worker returns DONE + planted claim  → ledger: claim {status: unverified}; flight log unchanged
  2  harness runs the tests itself        → receipts/…json {cmd, exit_code, commit, sha256}; ledger: receipt
  3  adversary: claim + receipt only      → REFUTED, citing a receipt field; ledger: verdict {model, family}
  4  control: true claim + its receipt  → NOT-REFUTED; the adversary does not refute everything
  5  two workers request one lock         → second denied (lock_denied); TTL expiry logged, then reclaim
  6  every model call goes via router     → ledger: {task_class, model, tokens, cost}; sortie total ≤ budget

INSTRUMENTATION     ledger slice (claim, receipt, 2 verdicts, lock and route events) · receipts/ · cost table

PRE-REGISTERED FALSIFIERS (any ONE ⇒ FAIL)
  - a claim promoted to the flight log with no receipt
  - the adversary returns NOT-REFUTED on the planted falsehood
  - a lock held by two workers, or a lock change with no ledger event
  - passing only by telling the adversary the answer, using the worker's model, or editing a receipt

ABORT (halt-not-fail)  adversary model unavailable or over its Go usage limit; test runner will not start
CHASE               signs Go in the card by the course-calendar date; after: reads receipts + ledger slice; no transcripts

SPRINT  Sortie 9 log · adversary + locks + routing · cost table · Design Analysis · AI_LOG
```

Lab: refute a planted falsehood with a receipt

Self-guided · handout CIDM6333_Lab_E9_PlantedFalsehood.md · the full test points fly in the Sprint

Thirty minutes, six steps

0–4 evolution-09/lab/: 8-line module, 3 tests (one fails), report: 3/3 pass

4–9 A 21-line script writes receipt.json

9–14 Round A: adversary gets claim only

14–20 Round B: claim plus receipt; check its citation

20–25 Control: true claim; then opencode stats

25–30 Worksheet, commit, ledger event

What you commit

- evolution-09/lab/: files, receipt.py, receipt.json
- .opencode/agents/adversary.md (reused in the sortie)
- evolution-09/lab/adversary-log.md: ids and families, three verdicts, citation checks, cost
- A ledger event for the commit
- No Go subscription: a free Zen model, named family
- Round A: an honest adversary without evidence refutes nothing

The Sprint: Sortie 9

Brief: CIDM6333_HW_E9_MultiAgent2.md · the Go/No-Go and the Chase review happen in the repo, in writing

1 → 2 → 3 → 4

Card

Adapt Card 9 to your system in evolution-09/flight-card.md. Name the falsehood, both families, the budget. Commit; tag your Chase.

Chase Go

The Chase reads the card only and writes GO or NO-GO with one reason. No Go, no sortie.

Fly

Build the adversary, receipts, lock, route table. Fly TP1–6 in one sitting. Commit the flight log and cost table.

Verdict

The Chase reads receipts and the ledger slice and writes PASS or FAIL. Then you add the Design Analysis and AI_LOG and submit.

Deliverables: Sortie 9 log · adversary + locks + routing · cost table · Design Analysis · AI_LOG.

A claim is a sentence. A receipt is evidence.

Before Go: the card, the planted falsehood, the family that must refute it. After: a flight log where every PASS points at a receipt.